Lab Manual: Foundations of Brain and Behavioral Research Workflows

1. Overview and the "Workflow Designer" Methodology

Welcome to the National Science Foundation Supported Learning Initiative (Award No. OAC-2417875). In this lab, you will not be memorizing programming syntax. Instead, you are assuming the role of a **Workflow Designer** —a professional who defines the logic and objectives of a research pipeline and collaborates with AI tools (such as Claude, ChatGPT, or GitHub Copilot) to execute the technical build.The core objective of this curriculum is to prioritize high-level workflow logic over the rote memorization of code. You will utilize the following three-step **Ask → Build → Document** cycle to complete all research activities:

1. **ASK** : You will provide a structured prompt to an AI tool. You are encouraged to adjust these prompts to better align with your specific research interests.
2. **BUILD** : You will take the code generated by the AI, execute it within the designated notebook cells, and verify the output against specific technical requirements.
3. **DOCUMENT** : You will provide a plain-language reflection on the results, specifically linking your observations to the foundational scientific and computing concepts found in your Reference Guide.

2. Laboratory Setup and Profile Initialization

Before beginning the simulation modules, you must initialize your environment. This ensures your research session is properly recorded.

- **Profile Setup** : Locate the "Profile Setup" cell and enter the required values for the following fields:
- **Student Name**
- **Institution**
- **Research Area** : Select from Neuroscience, Psychology, Biology, Engineering, or Other.
- **Initialization Confirmation** : Upon running the cell, the notebook must produce the following confirmation to be ready for use:

3. Key Terminology for Research Workflows

To ensure technical accuracy, you must adhere to the following verbatim definitions provided by the National Science Foundation support initiative:

- **Brain-Machine Interface (BMI)** : A system that connects the brain directly to an external device — bypassing damaged nerves or muscles — so that brain signals can control a robotic limb, cursor, or other tool.
- **fMRI** : Functional Magnetic Resonance Imaging — a brain scanning method that measures blood oxygen levels as a stand-in for neural activity. When neurons become active, they consume more oxygen, causing a detectable change in the local blood signal.
- **Working Memory (RAM) vs. Permanent Storage** : RAM (working memory) holds data only while the computer is on — it is extremely fast but temporary. The hard drive stores files permanently but is much slower to access.

- **Decoder** : A mathematical or statistical model that translates continuous brain signal patterns into a useful output — such as movement direction, cursor position, or speech intent.

4. Module 1: Brain Data Capture and Simulation (Part A)

Activity A.1: The BMI Decoder Simulation

Provide the following prompt to your AI tool to simulate the translation of motor cortex signals into physical movement.

```
Act as a neuroscience research assistant helping a biology student
understand brain-machine interfaces. Write a Python script using NumPy
and Matplotlib that simulates how brain signals from the motor cortex
can be used to control a robotic limb. The script should:

1. Generate 2 seconds of synthetic data representing voltage
recordings from 64 brain electrodes, sampled 1000 times per second,
including realistic background noise and a few sudden signal spikes
representing muscle movements.
2. Write a simple decoding function that converts these raw voltage
patterns into X and Y position coordinates representing the intended
movement of a robotic limb.
3. Plot the raw electrode signals and the resulting decoded movement
path side-by-side. Add plain-language comments throughout explaining
what each step does and what the numbers represent.
```

Verification Checklist for Activity A.1:

- Does the script generate exactly 2 seconds of data?
- Are there 64 distinct electrode channels represented?
- Does the visualization show a side-by-side comparison of raw signals and the decoded path?

Activity A.2: Brain Imaging (fMRI) Simulation

Provide the following prompt to simulate the spatial and temporal resolution of functional brain imaging.

```
Act as a neuroscience instructor explaining fMRI to a biology student.
Write a Python script that builds a simplified simulation of a
functional brain imaging dataset. The script should:
1. Create a small 3D grid of brain regions (10 x 10 x 10 grid points),
each observed across 15 time points — representing a short fMRI
recording session.
2. Simulate a realistic brain activity pattern where a specific region
becomes active (increases its signal) and then returns to baseline,
mimicking what happens when a participant performs a task.
```

```
3. Add a simple head movement correction step: introduce a small
artificial shift in the data across time points, then show how a
correction routine detects and compensates for this.
4. Display a plot showing how the signal changes over time in an
active brain region compared to an inactive one. Use plain-language
comments to explain what each step represents biologically.
```

Verification Checklist for Activity A.2:

- Does the code define a 10x10x10 3D grid?
- Are there 15 distinct time points in the simulation?
- Does the output show a clear correction for head movement?

Part A Reflection Task

Reflect on the simulations in Activities A.1 and A.2 by answering the following:

1. **Data Architecture** : In the fMRI simulation, each 3D cube of tissue is known as a **Voxel** (Concept 6). How did the AI represent these voxels in the code's rows and columns?
2. **Signal Translation** : Explain what the **Decoder** (Concept 4) specifically did to the 64 channels of electrode data to make it useful for a robotic limb.
3. **Real-Time Constraints** : If there was a high **Signal Delay (Latency)** (Concept 3) between the motor cortex spikes and the limb movement, what would be the practical impact on the user?
4. **Data Integrity** : Why is **Head Movement Correction** (Concept 7) considered a mandatory step before a researcher can trust the activity recorded in a voxel?

5. Module 2: Computer Storage and Workflow Organization (Part B)

Activity B.1: RAM vs. Permanent Storage Performance

Provide the following prompt to test the physical speed limits of your computing environment.

```
Act as a computing instructor helping a biology researcher understand
how computers handle data. Write a Python script that demonstrates the
difference between working memory (RAM) and permanent storage (hard
drive). The script should:
1. Measure how long it takes to read and write a large array of one
million numbers directly in working memory versus saving the same data
to a temporary file on disk and reading it back.
2. Create a loop that gradually increases the size of a data array,
reporting how much working memory is being used at each step, and
printing a clear warning message when memory is getting full —
stopping safely before the computer runs out.
3. Print a clear summary comparing the speed of working memory versus
disk storage. Use plain-language comments throughout explaining what
is happening and why it matters for research workflows.
```

Verification Checklist for Activity B.1:

- Did the script provide a timing comparison for 1,000,000 numbers?

- Did the output display a "warning" message before a potential crash?
- Is the speed differential between RAM and Disk clearly stated?

Activity B.2: Designing Independent Workflow Stages

Provide the following prompt to demonstrate the importance of modular architecture in research software.

```
Act as a research software instructor. Write a Python script that
demonstrates why scientific workflows should be organized into
separate, independent stages — rather than one large block of code.
The script should:
1. Stage 1 (Obtaining Data): A function that loads or generates a set
of recorded sensor measurements. This function should not perform any
calculations or create any plots.
2. Stage 2 (Analyzing Data): A function that receives the data from
Stage 1 and calculates summary statistics (mean, standard deviation,
range). This function should not load data or create plots.
3. Stage 3 (Visualizing Results): A function that takes the results
from Stage 2 and creates a clean summary plot. Show clearly how these
three stages pass information to each other, and explain in comments
what would go wrong if all three were mixed together in a single block
of code.
```

Verification Checklist for Activity B.2:

- Are the three stages (Obtaining, Analyzing, Visualizing) separated into distinct functions?
- Does the code explicitly demonstrate information passing between stages?

Part B Reflection Task

Reflect on the computing constraints demonstrated in Module 2:

1. **The Crash Threshold** : Based on Activity B.1, describe what leads to a **Memory Overflow** (Concept 29). Why is it dangerous for a researcher to load an entire massive dataset into RAM at once?
2. **Architectural Logic** : Activity B.2 focused on **Keeping Stages Independent** (Concept 41). According to the AI's comments, what specific problems arise if you mix data-loading and visualization in a single, monolithic block of code?
3. **Performance Observations** : Quantify the speed difference you observed between Working Memory and Disk. How does this affect your decision on where to store data during an active analysis?

6. Integration and Self-Check

Complete the following table to confirm your understanding of how technical concepts map to your workflow activities.| Concept Area | Concept Number | Where You Demonstrated It (Cell ID) || ------ | ------ | ------ || Neuroscience | Concept 1 — BMI Purpose | || Neuroscience | Concept 3 — Signal Delay (Latency) | || Neuroscience | Concept 5 — fMRI Mechanics | || Computing |

Concept 26 — RAM vs. Permanent Storage | || Computing | Concept 36 — Organizing Code into Stages | || Computing | Concept 41 — Keeping Stages Independent | |

7. Technical Glossary

The following definitions are the foundational standards for this workshop:

- **Concept 1: Brain-Machine Interface (BMI)** : A system that connects the brain directly to an external device — bypassing damaged nerves or muscles — so that brain signals can control a robotic limb, cursor, or other tool.
- **Concept 3: Signal Delay (Latency)** : The time gap between a brain signal being recorded and a device responding to it. Delays longer than about 50–100 milliseconds feel unnatural to the user of a prosthetic device.
- **Concept 4: Decoder** : A mathematical or statistical model that translates continuous brain signal patterns into a useful output — such as movement direction, cursor position, or speech intent.
- **Concept 5: fMRI** : Functional Magnetic Resonance Imaging — a brain scanning method that measures blood oxygen levels as a stand-in for neural activity. When neurons become active, they consume more oxygen, causing a detectable change in the local blood signal.
- **Concept 6: Voxel** : A small 3D cube of brain tissue in an fMRI image — the brain-imaging equivalent of a pixel. Each voxel contains millions of neurons.
- **Concept 7: Head Movement Correction** : A processing step that aligns brain scan images across time, compensating for small movements the participant made during the recording session.
- **Concept 26: Working Memory (RAM) vs. Permanent Storage** : RAM (working memory) holds data only while the computer is on — it is extremely fast but temporary. The hard drive stores files permanently but is much slower to access.
- **Concept 29: Memory Overflow** : What happens when a script tries to load more data into working memory (RAM) than the computer has available — causing the program to crash.
- **Concept 36: Organizing Code into Stages** : Separating a workflow into clearly defined, independent stages — such as one stage for loading data, one for analysis, and one for visualization — makes it much easier to find and fix problems.
- **Concept 39: Settings Files (JSON/YAML)** : Lightweight text files used to store configuration settings, metadata, and parameters for a workflow — making it easy to share, reproduce, or adjust an analysis without changing the code itself.
- **Concept 41: Keeping Stages Independent** : Designing a workflow so that the data analysis stage does not depend on the visualization stage, and vice versa. This means you can update or replace one stage without breaking the others.